UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 2: Introduction to Classes and Objects

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

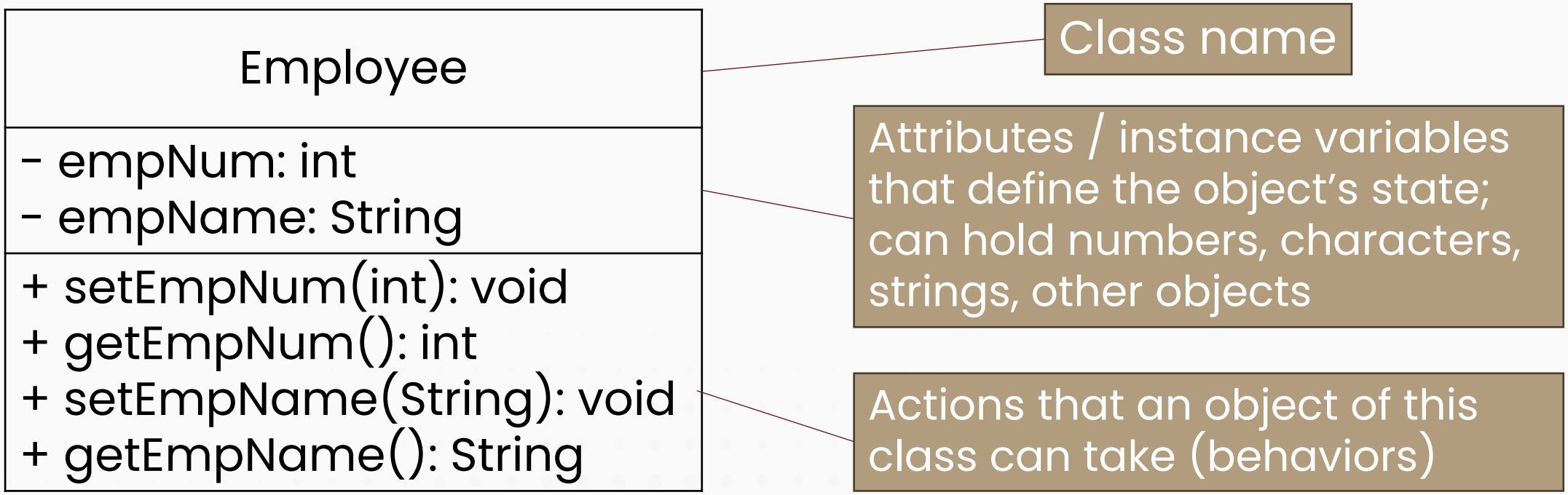
Contents

- **Creating Class and Object:**
  - Instance Fields
  - Accessor and Mutator Methods
  - Constructors
- **Passing Arguments to Methods**
- **Static Variable and Method**

Classes

- A Java class member may have:
  - **instance variables** to define data fields/attributes and
  - **instance methods** to define behaviors
- A class provides a **special type of methods**, known as **constructors**
- Constructors are invoked to construct objects from the class

UML Class Diagram: Example Employee Class



Employee Class Shell

- A "class shell" refers to the **basic structure** or framework of a class in programming, initially defined without much detail or implementation.
- It is a **skeletal outline of a class** that includes just the essential elements, e.g. the class name, its attributes (variables), and method placeholders, **without any complex logic**, behavior, or functionality.

```java
public class Employee {
    // Attributes (instance variables)
    private int empNum;
    private String empName;

    // Constructor
    public Employee() {
        // Default constructor
    }

    // Method placeholders
    public void setEmpNum(int empNum) {
        // Logic to be added later
    }

    public int getEmpNum() {
        // Logic to be added later
        return empNum;
    }
}
```

Data Fields

- Variables declared within a class but **outside of any methods**
- Called as **instance variables** (local) when not static (global)
- Each class will have own copy of each non-static data field
- `private` access for fields:
  - No other classes can access field's values
  - Only methods of same class allowed to use private variables

Data Fields: Example

```
private int empNum;
Private String empName;
```

private [static] [final] *datatype name;*

- Usually private
- If stated, it means the field is **shared by all objects** in the class
- If stated, it means the field is a **constant**
- Primitive e.g. int, double or an object e.g. String
- Given name

Creating Instance Methods in a Class

- Classes contain methods
- **Non-static methods** are instance methods **belong to objects**
- Typically declare non-static instance variables
- `static` class variables are not instance variables
- Use **"this"** when **referring to an instance variable**, differs from the method's parameter (if we use the same name)

```java
public class Example {
    // Instance variable
    private String message;

    // Constructor to initialize the instance variable
    public Example(String message) {
        this.message = message;
    }

    // Non-static method
    public void displayMessage() {
        System.out.println("Message: " + message);
    }

    public static void main(String[] args) {
        // Creating an object of the Example class
        Example obj = new Example("Hello, World!");

        // Calling the non-static method using the object
        obj.displayMessage();
    }
}
```

Method Definitions

- Method header form:

```
modifier returnType methodName ([parameterList])
```

- Example:

```
public double getPrice()
public void setPrice(double aPrice)
```

- Method names usually start with verbs i.e. **verb-noun pairs**
- The name of a method that **returns the value** of a field often starts with `get`: e.g. `getWidth, getX`
- The name of a method that **sets the value** of a field often starts with `set`: e.g. `setLocation, setText`

Data Hiding via `Set` and `Get` Methods

- **Data hiding** using encapsulation:
  - Data fields usually **`private`**
  - Client application accesses them only through **`public`** interfaces
- **`Set` method** (setter): Controls the update of values for private data fields
- **`Get` method** (getter): Controls how values are retrieved

```java
public class Employee {
    // Private instance variable
    private int empNum;

    // Setter for empNum
    public void setEmpNum(int empNum) {
        this.empNum = empNum;
    }

    // Getter for empNum
    public int getEmpNum() {
        return empNum;
    }

    public static void main(String[] args) {
        // Create an Employee object
        Employee emp = new Employee();

        // Set the employee number
        emp.setEmpNum(101);

        // Get and display the employee number
        System.out.println("Employee Number: " + emp.getEmpNum());
    }
}
```

Accessor and Mutator Methods

- Methods that **retrieve** data fields are called **accessors**
- Methods that **modify** data fields are called **mutators**
- E.g. For a rectangle, the accessors and mutators are:
  - **`getLength`** : Returns the value of the length field

```
public double getLength() { }
```

  - **`getWidth`** : Returns the value of the width field

```
public double getWidth() { }
```

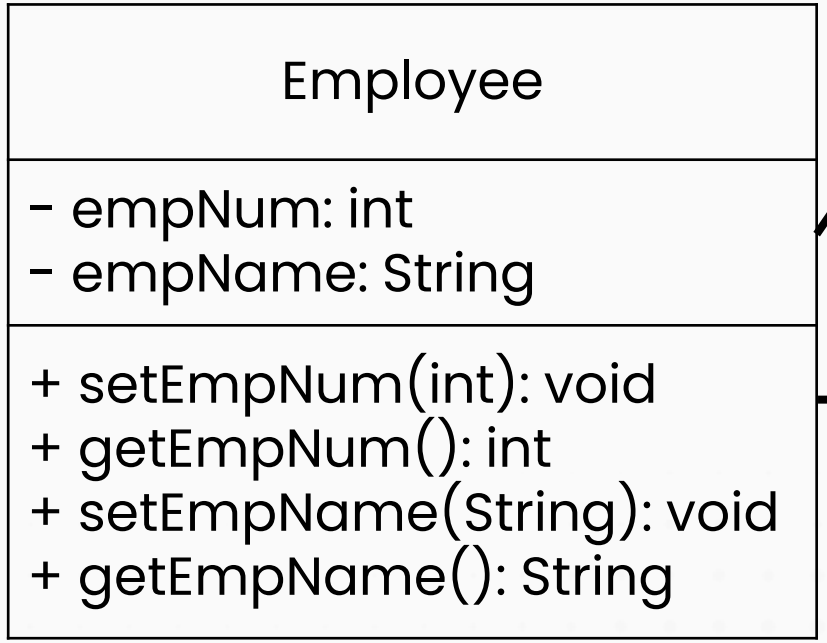
  - **`setLength`** : Sets the value of the length field

```
public void setLength(double len) { }
```

  - **`setWidth`** : Sets the value of the width field

```
public void setLength(double w) { }
```

Class Definition




```java
public class Employee {
  private int empNum;
  private String empName;

  public void setEmpNum(int num) {
   empNum = num;
  }

  public int getEmpNum() {
    return empNum;
  }

  public void setEmpName(String name) {
    empName = name;
  }

  public String getEmpName() {
    return empName;
  }
}
```

Note: This class `Employee` follows the Unfied Modelling Language (UML) for a class diagram

Declaring Objects or Create an Instance

- **Define a class** does not create any actual objects, only the **template**
- Create **instance of class**:
  - Supply type and identifier
  - Allocate computer memory for object

```java
// Define the Employee class
class Employee {
    String name;
    int id;

    // Constructor to initialize the instance
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Method to display employee details
    void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + id);
    }
}

public class Main {
    public static void main(String[] args) {
        // Create an instance of the Employee class
        Employee emp1 = new Employee("Alice", 101);

        // Use the object
        emp1.displayDetails();
        // Output:
        // Employee Name: Alice
        // Employee ID: 101
    }
}
```

Declare Object Reference Variable and Create Object

- To refer an object, assign the object to a reference variable.
- To **declare a reference variable**, use the syntax:

```
ClassName objectRefVar;
```

- To create object:

```
objectRefVar = new ClassName();
```

- Example:

```
Employee emp1;
emp1 = new Employee();
```

Declare and Create Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Assign object reference

Example:

```
Employee emp1 = new Employee();
```

Create an object

Accessing Objects' Methods

- After object instantiated, methods are accessed using:
  - Object's identifier
  - Dot
  - Data/Method call
- Referencing the object's data:

```
objectRefVar.data
```

- Example:

```
emp1.empNum
```

- Invoking the object's method:

```
objectRefVar.methodName(arguments);
```

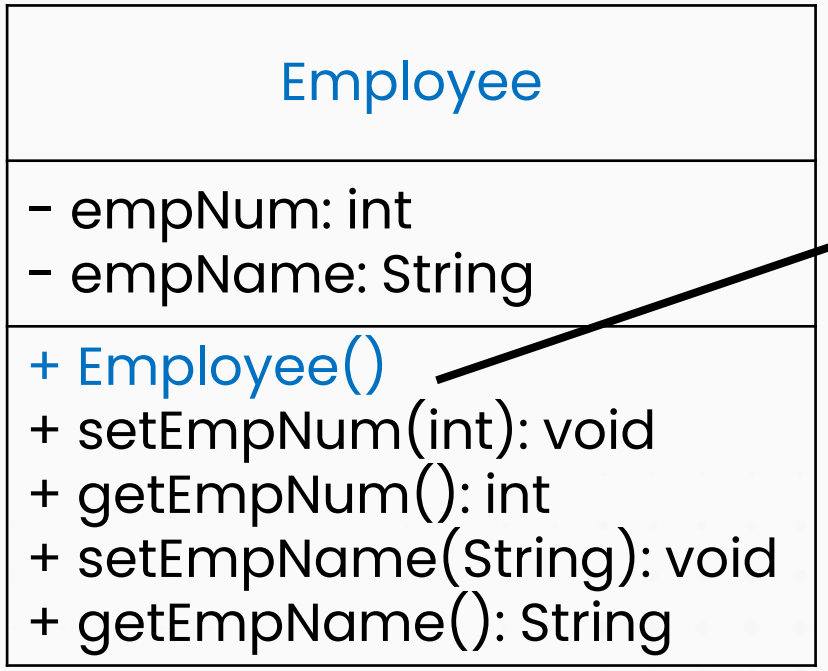
- Example:

```
emp1.getEmpNum();
```

Constructors

- Special kind of **methods** for **creating objects** of the class.
- A class may be declared without constructors.
- If a programmer does not define any constructors, Java will automatically provides one **default no-args constructor**, which allocates memory and sets fields to the default values
- Constructor name are always the same as class name.
- Default constructors **do not have a return type** (not even void) and they do not return a value.
- Example: `public Employee() { }`

Class Definition with Default Constructor




```java
public class Employee {
  private int empNum;
  private String empName;

  public Employee() {
    System.out.println ("Start Employee");
  }
  public void setEmpNum(int num) {
   empNum = num;
  }
  public int getEmpNum() {
    return empNum;
  }
  public void setEmpName(String name) {
    empName = name;
  }
  public String getEmpName() {
    return empName;
  }
}
```

Creating Object, Invoke Constructors

- Constructors may take parameters.
  - If a constructor has one parameter, it is a **one-argument constructor**
- If a class has **more than one** constructor, they must have different numbers and/or types of parameters: **constructor overloading**
- Constructors are **invoked using the `new` operator** when an object is created, they play the role of **initializing objects**.
- Format: `new ClassName();`
- Example:

```
new Employee();
new Employee(123, "Ahmad");
```

No-Arg Constructor: Example

- A no-arg (no-argument) constructor is a constructor **that does not take any parameters**.
- It is used to **create objects with default values** or perform basic initialization.
- If any constructors are not explicitly defined in a class, Java automatically provides a default no-arg constructor.

```java
// Class representing an Employee
class Employee {
    String name;
    int id;

    // No-arg constructor: initializes default values
    Employee() {
        name = "Unknown";
        id = 0;
    }

    // Method to display employee details
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object using the no-arg constructor
        Employee emp = new Employee();

        // Display details of the object
        emp.displayDetails();
    }
}
```

Copy Constructor: Example

- A copy constructor is a constructor that creates a new object by **copying the attributes** of an existing object.
- This is particularly useful to **duplicate an object** with its current state.

```java
// Class representing an Employee
class Employee {
    String name;
    int id;

    // Parameterized constructor: initializes attributes
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Copy constructor: creates a copy of an existing object
    Employee(Employee existingEmployee) {
        this.name = existingEmployee.name;
        this.id = existingEmployee.id;
    }
    // Method to display employee details
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("ID: " + id);
    }
}
public class Main {
    public static void main(String[] args) {
        // Creating the original object using the parameterized cons
        Employee original = new Employee("Alice", 101);

        // Creating a copy of the original object using the copy cor
        Employee copy = new Employee(original);

        // Display details of both objects
        original.displayDetails();
        copy.displayDetails();
    }
}
```

Constructor: Bug Example

- A **constructor** must have the same name as the class and cannot include a return type (not even `void`)
- Otherwise, it **becomes a regular method** not as a constructor

```
public class MyWindow extends JFrame
{
    ...
    // Constructor:
    public  void  MyWindow ( )
    {
        ...
    }
    ...
```

Compile is fine, but the compiler thinks this is a method and uses **MyWindow**'s default no-args constructor instead.

Constructor Chaining

- Constructors of a class can **call each other** using the keyword `this`
- A good way to avoid duplicating code, known as **constructor chaining**
- The no-arg constructor `Employee()` calls the single-argument constructor `Employee(String name)` using `this("Unknown")`

```java
class Employee {
    String name;

    // No-arg constructor
    Employee() {
        this("Unknown");
    }

    // Constructor with one argument
    Employee(String name) {
        this.name = name;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee emp = new Employee(); // Calls no-arg constructor
        emp.displayDetails(); // Output: Name: Unknown
    }
}
```

Employee.java

```
import …

public class Employee
{
```

- **Fields**
- **Constructors**
- **Methods**

```
}
```

Import statements

Attributes / variables that define the object's state; can hold numbers, characters, strings, other objects

Procedures for constructing a new object of this class and initializing its fields

Actions that an object of this class can take (behaviors)

Lab Exercise: Identify Errors

CREATING CLASS & OBJECT

```
public class Circle {
  private double radius, area;
  final PI = 3.14159;

  Circle (double r) {
    radius = r;
  }
  public double findArea() {
    area= radius*radius*PI;
    return area;
  }
}

public class TestCircle {
  public static void main(String[] args) {
    Circle circle1 = new Circle(2.3);
    System.out.println("The area of the circle of radius "+ circle1.radius + "
is " + circle1.findArea());
  }
}
```

File Name?
Save as "____.java"
Identify 3 to 4 errors
and correct them.

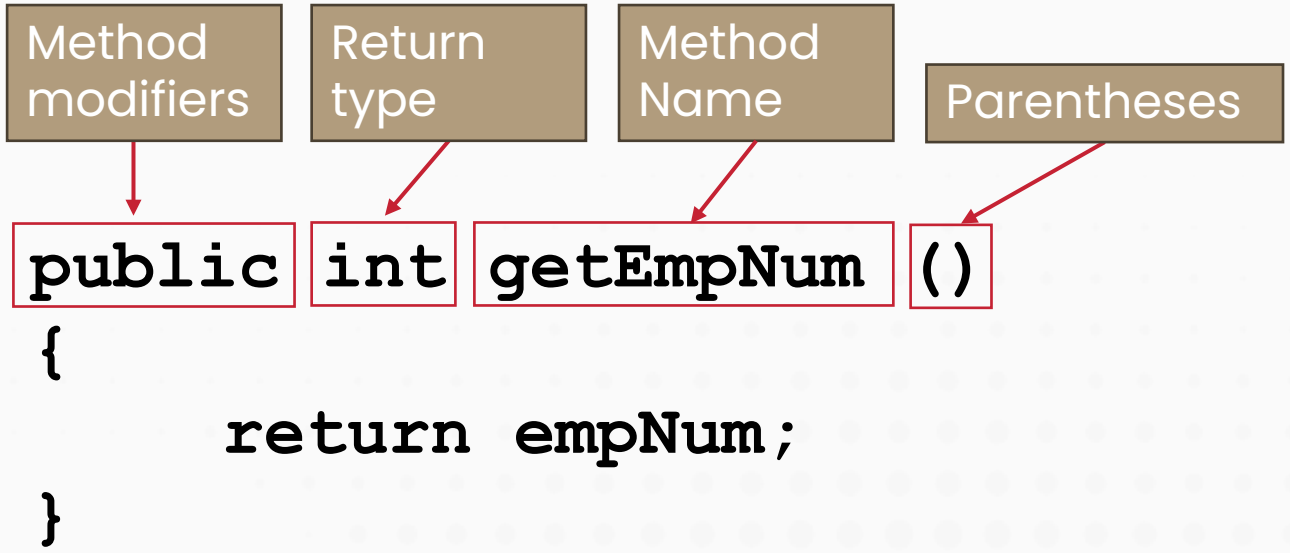
Method

- A method is a **collection of statements** that are grouped together to perform an operation.
- A method is always defined **inside a class**.
- **Method signature or method header** is the combination of the method name and the **parameter list**.

```
Method definitions have the form:
method header {
        method body
}
```

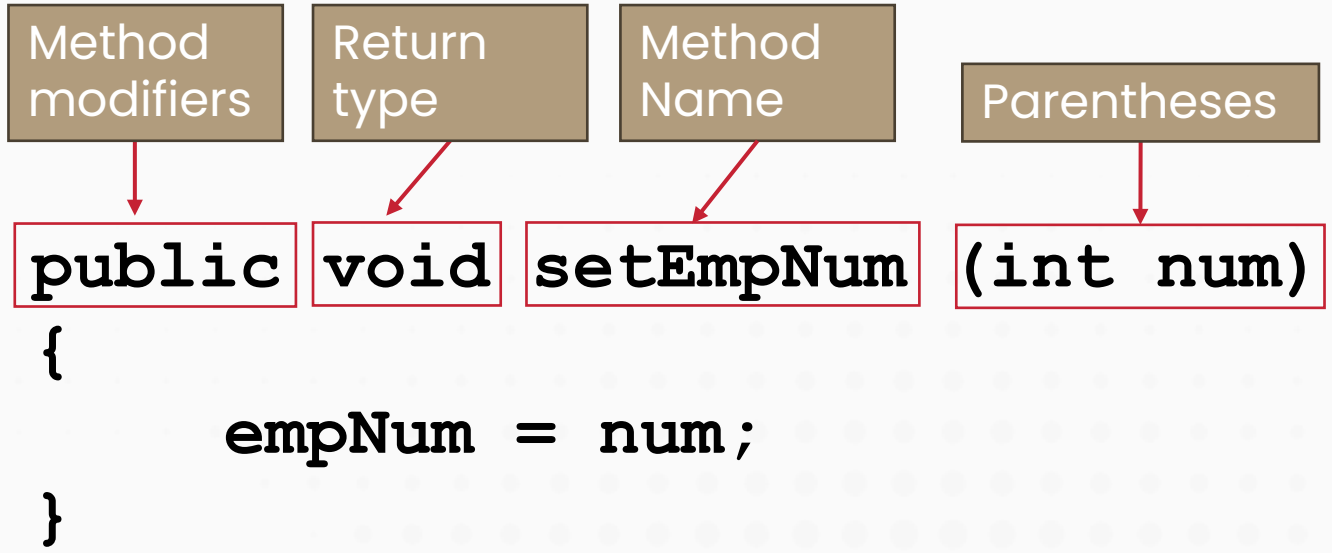
Parts of a Method Header: Example `int` Return Type

- The method below is an **instance method**.
- The type of result returned is return type where `int` means the method returns an integer result.



Parts of a Method Header: Example `void` Return Type

- The method below is an **instance method**.
- The type of result returned is return type where `void` means the method does not return a result.



`return` Statement

- A method, unless `void`, returns a value of the specified type to the **calling method**.
- The `return` statement is used to immediately quit the method and return a value
- A method **can have several `return` statements**; then all but one of them must be inside an `if` or `else` (or in a `switch`)

return *expression;*

The type of the return value or expression must match the method's declared return type.

```
public someType myMethod (...)
{
    ...
    if (...)
        return <expression1>;
    else if (...)
        return <expression2>;
    ...
    return <expression3>;
}
```

`return` Statement: Example for `boolean`

- A `boolean` return type method can return true, false, or the result of a `boolean` expression
- Example:

```
public boolean myMethod (...)
{
    ...
    if (...)
        return true;
    ...
    return n % 2 == 0;
}
```

Calling an Instance Method

- A method **executes** when it is **called**.
- The `main` **method** is automatically called when a **program starts**, but other methods are executed by method call statements.
- A method with a **non-void return value type** can also be invoked as a statement.
- An invocation of an **instance method** that returns an integer value may use an assignment statement or output statement:

```
int vnum;
vnum = obj1.getEmpNum();
```

or

```
System.out.println ("employee num:" + obj1.getEmpNum());
```

Calling a `void` Instance Method

- An invocation of a `void` instance method is simply a statement:

```
objectName.methodName();
```

- Example:

```
obj1.setEmpNum(123);
```

Passing Parameters to Constructors

- Any **expressions** that have an appropriate data type can serve as a parameter

```
 public static void main (String[] args) {
   Employee emp1 = new Employee();
   Employee emp2 = new Employee(123); // argument
   Employee emp3 = new Employee(456, "Ahmad"); // argument
}
```

```
public class Employee {
   public Employee () {  }
   public Employee (int num) {  // parameters
        empNum = num;
   }
   public Employee (int num, String name) {
        empNum = num;
        empName = name;
   }
}
```

Passing Arguments to Method

- Values that are sent into a method are called **arguments** e.g.

```
Scanner input = new Scanner(System.in);
Employee obj1 = new Employee();
System.out.print("Input the employee number:");
int number = Integer.parseInt(input.nextline());
obj1.setEmpNum(number);
```

- The **data type** of an **argument in a method call** must **correspond to the variable declaration** in the parentheses of the method declaration.
- The **parameter** is the variable that holds the value being passed into a method.

Arguments are Passed by Value

- In Java, all arguments of the primitive data types are ***passed by value***: only a **copy of an argument's value** is passed into a parameter variable.
- Any change to a **parameter variable** inside a method, **do not affect** the original argument.

```java
public class PassByValueExample {
    public static void main(String[] args) {
        int number = 10; // Original value
        System.out.println("Before method call, number: " + number);

        modifyValue(number); // Passing the value to the method
        System.out.println("After method call, number: " + number);
    }

    public static void modifyValue(int num) {
        num = num * 2; // Modifies the copy, not the original
        System.out.println("Inside method, num: " + num);
    }
}
```

Passing Parameters : Primitive Data Type

- Primitive data types are always passed "by value": the **value is copied** into the parameter

```
int x = 123;
emp1.setEmpNum (x);
```

copy

```
public class Employee
{
    ...
    public void setEmpNum (int num)
    {
        empNum = num;
        ...
    }
}
```

num acts like a **local variable** in setEmpNum

x: 123 —copy→ num: 123

Passing Parameters: Example

```
public class Test
{
    public double square (double x)
    {
        x *= x;    //x = x * x
        return x;
    }

    public static void main(String[ ] args)
    {
        Test calc = new Test ();
        double x = 3.0;
        double y = calc.square (x);
        System.out.println (x + "  " + y);
    }
}
```

x here is a copy of the parameter passed to **square**. The copy is changed, but…

… the original **x** is unchanged. Output: **3 9**

The value of x in main does not change, because **Java passes primitive types by value,** meaning the method receives a **copy of x**, not the actual variable

Passing Objects as Arguments

- Objects can be passed to methods **as arguments**.
- Java passes all arguments ***by value***.
- When an object is passed as an argument, the **value of the reference variable** is passed.
- The value of the reference variable is an **address or reference to the object in memory**.
- When a method receives a reference variable as an argument, it is possible for the method to **modify the contents** of the object referenced by the variable.

Passing Objects as Arguments to Methods

PASSING ARG. TO METHODS

```
public class Rectangle {
  private double width;
  private double length;

  public Rectangle (double w, double l) {
    width = w;
    length = l;
  }

  public void setWidth(double w) {
    width = w;
  }

  public void setLength(double l) {
    length = l;
  }
```

```
  public double getWidth() {
     return width;
   }

   public double getLength() {
     return length;
   }

   public double calcArea() {
     return width * length;
   }

   public boolean equals(Rectangle r) {
      return (this.calcArea() == r.calcArea());
   }
}
```

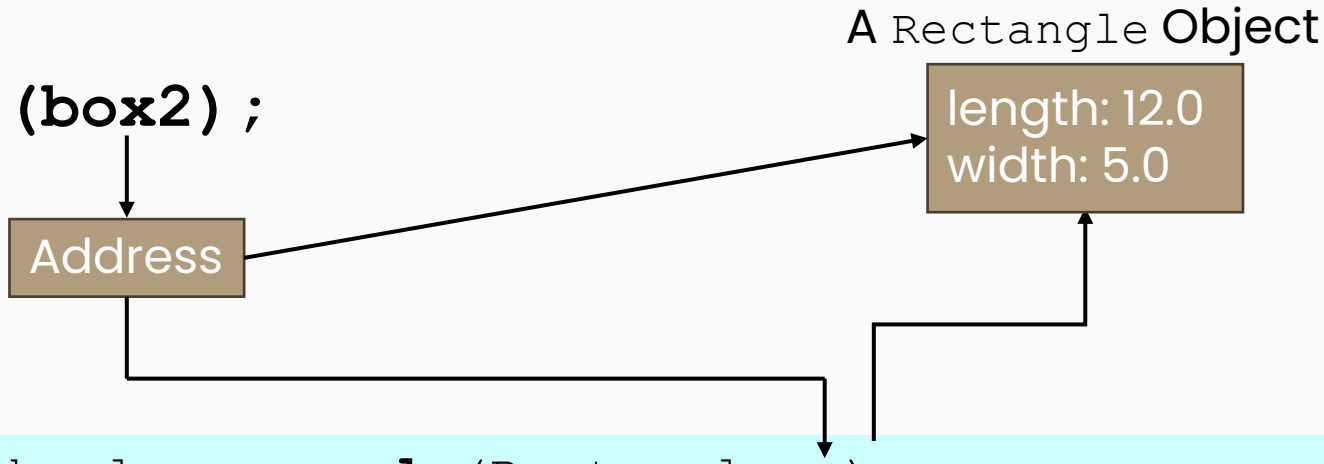
Passing Objects as Arguments to Methods

```
1. public class PassObject  {
2.    public static void main(String[ ] args)  {
3.       Rectangle box1 = new Rectangle(12.0, 5.0);
4.       Rectangle box2 = new Rectangle(10.0, 8.0);
5.       if (box1.equals(box2))
6.          System.out.println ("Yes, it's equal");
7.       else
8.          System.out.println ("No, it's not equal");
9.     }
10. }
```

- Line 3 and 4 creates two instances of `Rectangle` class.
- Line 5: the value of the variable box2 (reference variable of `Rectangle`) is passed as an argument to the instance method, `equals()`.
- In Rectangle class, equals() method has a parameter, `r`, which is a Rectangle reference variable and return a `boolean` value, after comparing whether the two objects are equal.

Passing Objects as Arguments

```
box1.equals(box2);
```




```
public boolean equals(Rectangle r)
{
    return (this.calcArea() == r.calcArea());
}
```

Note:
`box1` is the **object you're calling the method on** — this becomes the `this` reference inside the equals method
`box2` is the **object being passed in** — this becomes the parameter (usually obj) inside the method.

Passing Objects from Methods

```
import java.util.*;

public class PassObject3 {

   public static void main(String[ ] args) {
      Rectangle box3 = new  Rectangle().inputRectangle();
      System.out.println ("The area: " + box3.calcArea());
   }

   public Rectangle inputRectangle() {
      Scanner inp = new Scanner (System.in);
      double newWidth, newLength;

      System.out.print ("Enter width : ");
      newWidth = inp.nextDouble();
      System.out.print ("Enter length : ");
      newLength = inp.nextDouble();

      return new Rectangle(newWidth, newLength);
   }
}
```

Returning Objects From Method

- Methods can return **references to objects**.
- Just as with passing arguments, a copy of the object is not returned, only its **address**.
- Method return type e.g.

```
public Rectangle inputRectangle()
{
    ...
    return new Rectangle(newWidth, newLength);
}
```

toString Method

- All objects have a `toString` method that **returns the class name** and a **hash of the memory address** of the object.
- Can **override the default method** with our own to print out more useful information.
- In Java, all classes implicitly inherit from java.lang.Object, and Object provides a **default implementation of the toString()** method.

```
class Example {
    public static void main(String[] args) {
        Example exampleObject = new Example();
        System.out.println(exampleObject.toString());
    }
}
```

Output:

```
Example@6d06d69c
```

```
class Rectangle {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public String toString() {
        return "Rectangle[Width=" + width + ", Height=" + height + "]";
    }
    public static void main(String[] args) {
        Rectangle rect = new Rectangle(5, 10);
        System.out.println(rect.toString()); // Outputs
    }
}
```

`equals` Method

- When the **==** **operator** is used with **reference variables**, the **memory addresses** of the objects are compared not the contents
- All objects have an **`equals`** method.
- The **default operation of the `equals` method** is to compare memory addresses of the objects (just like the **==** operator).
- The **`Rectangle`** class has an **`equals`** method.
  - Write a method that only the addresses of the objects are compared
- Instead of using the **==** operator to compare two **`Rectangle`** objects, we should use the **`equals`** method.
  - **Compare objects by their contents** rather than by their memory addresses

`equals` Method: Example

- The Rectangle class has an equals method.
- Try the following e.g.

```
Stock stock1 = new Stock("GMX", 55.3);
Stock stock2 = new Stock("GMX", 55.3);
   if (stock1 == stock2) // This is a mistake
         System.out.println("The objects are the same.");
   else
         System.out.println("The objects are not the same.");
```

Note: Only the **addresses of the objects are compared**, though the contents are equal i.e. 55.3

equals Method: Use @Override equals

PASSING ARG. TO METHODS

```java
public class Stock {
    private String symbol;
    private double price;

    public Stock(String symbol, double price) {
        this.symbol = symbol;
        this.price = price;
    }

    // Override equals to compare contents
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;

        Stock stock = (Stock) obj;
        return Double.compare(stock.price, price) == 0 && symbol.equals(stock.symbol);
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Stock stock1 = new Stock("GMX", 55.3);
        Stock stock2 = new Stock("GMX", 55.3);

        if (stock1.equals(stock2)) {
            System.out.println("The objects have the same content.");
        } else {
            System.out.println("The objects are not the same.");
        }
    }
}
```

Methods that Copy Objects

- Cannot use the assignment operator to copy reference types
- There are **two ways** to copy an object:
  - **Reference only copy (shallow copy):** This is simply **copying the address** of an object into another reference variable.
  - **Deep copy (correct way):** This involves creating a new instance of the class and **copying the values** from one object into the new object

Reference Copy vs. Deep Copy

```java
Stock stock1 = new Stock("GMX", 55.3);
Stock stock2 = stock1; // Just copies the reference

stock2.setPrice(60.0); // Changing stock2 also changes stock1!

System.out.println(stock1.getPrice()); // Outputs: 60.0
```

Reference Copy: `stock1` and `stock 2` **refer to the same object**

```java
public class Main {
    public static void main(String[] args) {
        Stock stock1 = new Stock("GMX", 55.3);
        Stock stock2 = new Stock(stock1); // Deep copy

        stock2.setPrice(60.0); // Only changes stock2

        System.out.println("Stock1 price: " + stock1.getPrice()); // 55.3
        System.out.println("Stock2 price: " + stock2.getPrice()); // 60.0
    }
}
```

```java
public class Stock {
    private String symbol;
    private double price;

    public Stock(String symbol, double price) {
        this.symbol = symbol;
        this.price = price;
    }

    // Copy constructor (Deep copy)
    public Stock(Stock other) {
        this.symbol = other.symbol;
        this.price = other.price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public String getSymbol() {
        return symbol;
    }
}
```

Deep Copy: create a **new object** and copy the contents

Copy Constructors: Deep Copy Example

- A copy constructor accepts an existing object of the same class and clones it e.g.

```
public Rectangle(Rectangle object2)
{
      this.width = object2.width;
      this.length = object2.length;
}

// Create a rectangle object, rect1
Rectangle rect1 = new Rectangle(5.0, 4.5);

//Create rect2, a copy of rect1
Rectangle rect2 = new Rectangle(rect1);
```

FoodItem Class

- The class represents a food item at a market with **three attributes** and **nine methods** as below

FoodItem
- description: String - size: double - price: double
+ FoodItem() + setDesc() + setSize() + setPrice() + getDesc() + getSize() + getPrice() + calcUnitPrice() + toString()

Methods for Class `FoodItem`

For most classes, we need to provide methods that perform tasks similar to the ones below:

- A method to **create a new object** in this e.g. FoodItem object with a specified description, size, and price (*constructor*)
- Methods to **change the attributes**: description, size, or price of an existing FoodItem object (*mutators*)
- Methods that **return the attributes**: description, size, or price of an existing FoodItem object (*accessors*)
- A method that **returns the object state** as a string (a `toString()` method)

Methods Headers for Class `FoodItem`

Method	Description
`public FoodItem(String, double, double)`	Constructor - creates a new object whose three data fields have the values specified by its three arguments.
`public void setDesc(String)`	Mutator - sets the value of data field **description** to the value indicated by its argument. Returns no value.
`public void setSize(double)`	Mutator - sets the value of data field **size** to the value indicated by its argument. Returns no value.

Methods Headers for Class `FoodItem`

Method	Description
`public void setPrice(double)`	Mutator - sets the value of data field **price** to the value indicated by its argument. Returns no value.
`public String getDesc()`	Accessor - returns a reference to the string referenced by data field **description**.
`public double getSize()`	Accessor - returns the value of data field **size**.
`public double getPrice()`	Accessor - returns the value of data field `price`.
`public String toString()`	Returns a string representing the state of this object.
`public double calcUnitPrice()`	Returns the unit price (price / size) of this item.

Class FoodItem

```
/*
 * FoodItem.java
 * Represents a food item at a market
 */

public class FoodItem {

  // data fields
  private String description;
  private double size;
  private double price;
```

Methods in Class `FoodItem`

```
// methods
  // postcondition: creates a new object with data
  // field values as specified by the arguments
  public FoodItem(String desc, double aSize, double aPrice) {
     description = desc;
     size = aSize;
     price = aPrice;
  }

  // postcondition: sets description to argument value
  public void setDesc(String desc) {
     description = desc;
  }
```

Methods in Class `FoodItem`

```
// postcondition: sets size to argument value
public void setSize(double aSize) {
   size = aSize;
}

// postcondition: sets price to argument value
public void setPrice(double aPrice) {
   price = aPrice;
}

// postcondition: returns the item description
public String getDesc() {
   return description;
}
```

Methods in Class `FoodItem`

```
  // postcondition: returns the item size
  public double getSize() {
     return size;
  }
  // postcondition: returns the item price
  public double getPrice() {
     return price;
  }
  // postcondition: returns string representing the item state
  public String toString() {
     return description + ", size : " + size + ", price $" + price;
  }
  // postcondition: calculates & returns unit price
  public double calcUnitPrice() {
     return price / size;
  }
}
```

void Methods

- The mutator methods are void methods:

```
public void setSize(double aSize) {
    size = aSize;
}
```

- Method `setSize()` sets the value of data field size to the value passed to parameter `aSize`.
- A void method is executed for its effect because it **does not return a value**.
- However, it may change the state of an object or cause some information to be displayed.

`return` Statement

- Each **`FoodItem`** method that is not a **`void`** method or a constructor returns a single result.

```
public double getPrice() {
        return price;
}
```

The **`return`** statement above **returns the value of data field `price`** as the function result.

- Form: **`return`** expression;
- Example: **`return x + y;`**
- Interpretation: The expression after the **keyword `return`** is returned as the method result. Its data type must be assignment compatible with the type of the method result.

`toString()` Method

- A **`toString()`** method returns a reference to a **`String`** object that **represents an object's state**.
- The expression after **`return`** specifies the characters stored in the **`String`** object returned as the method result.
- It starts with the characters in **`description`** followed by the characters **`size`** followed by characters representing the value of data field **`size`** and **`price`** e.g.

  `"apples, Macintosh, size: 2.5, price $1.25"`

```
public String toString() {
    return description + ", size : " + size + ", price $" + price;
}
```

Calling Methods

- Because a **`void`** method does not return a value, a call to a void method is a Java statement all by itself. If **`myCandy`** is type **`FoodItem`**, an example would be:

```
myCandy.setDescription("Snickers, large");
```

- Because a non-**`void`** method returns a value, a call to a non-**`void`** method must occur in an expression:

```
"price is $" + myCandy.getPrice()
```

- The result returned by the call to method **`getPrice()`** of type **`double`** is appended to the string that starts with the characters **`price is $`**

Call to Constructor

```
public FoodItem(String desc, double aSize, double aPrice) {
    description = desc;
    size = aSize;
    price = aPrice;
}
```

- A constructor executes when a **new object is created and initializes** the object's data fields.
- A call to this constructor always follows the word `new` and has the form:

```
FoodItem myCandy = new FoodItem("Snickers", 6.5, 0.55);
```

Argument/Parameter Correspondence for `FoodItem("Snickers", 6.5, 0.55)`

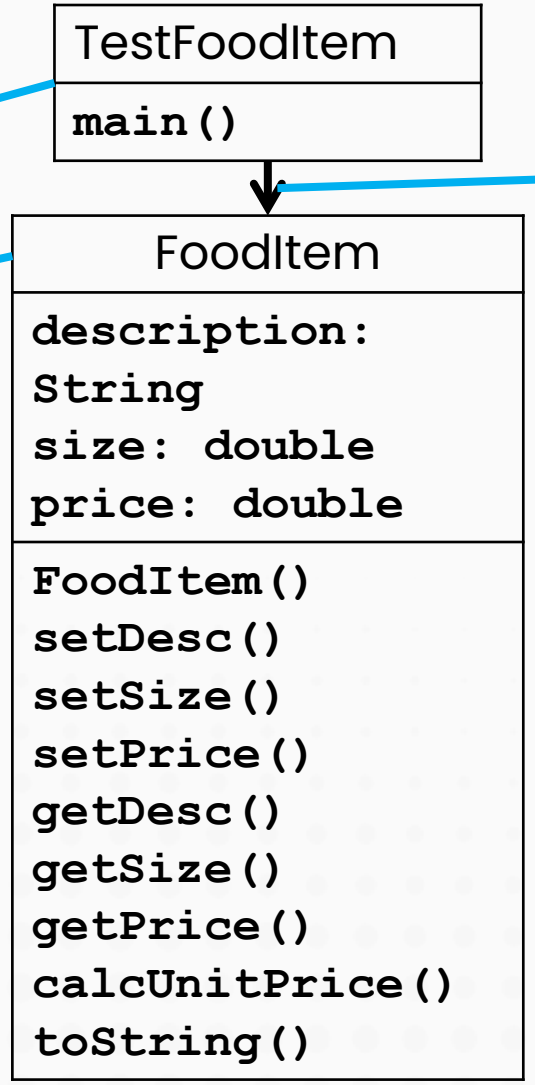
Argument	Parameter
"Snickers"	desc (references "Snickers")
6.5	aSize (value is 6.5)
0.55	aPrice (value is 0.55)

Primitive type arguments are **passed by value** which means the method cannot change the argument value.

Testing Class FoodItem

client (application) class

worker class



association relationship

Program classes and their relationships

Application or Client of FoodItem

```
/*
 * TestFoodItem.java
 * Tests class FoodItem
 */
public class TestFoodItem {

  public static void main(String[] args) {
    FoodItem myCandy = new FoodItem("Snickers", 6.5, 0.55);
    FoodItem mySoup = new FoodItem("Progresso Minestrone", 16.5, 2.35);
    System.out.println(myCandy.toString());
    System.out.println(" -- unit price is $" + myCandy.calcUnitPrice());
    System.out.println(mySoup.toString());
    System.out.println(" -- unit price is $" + mySoup.calcUnitPrice());
  }
}
```

Sample Run of Class TestFoodItem

TestFoodItem

```
Snickers, size : 6.5, price $0.55
 -- unit price is $0.08461538461538462
Progresso Minestrone, size : 16.5, price $2.35
 -- unit price is $0.14242424242424243
```

Static Modifier

- The static modifier is used to create variables and methods that will **exist independently** of any instances created for the class.
- In other words, **static members exist before creating a new instance of a class.**
- There will be **only one copy of the static member**, regardless of the number of instances of that class.

Reasons for Using Static Modifier

1. Need to have **a method that always runs** the same way without having dependency on the instance variable values: **ask the class itself to run the method**, and not instance-specific.
2. Need to **count all instances instantiated from a particular class**: It will not work to keep it as an instance variable within the class whose instances are being tracked as the count will just be initialized back to a default value with each new instance.
3. Variables and methods marked static **belong to the class** rather than to any particular instance.
4. Need to have the **class available to be able to invoke** a static method or access a static variable.

Reasons for Using Static Modifier (cont'd)

5. Static variables, too, can be **accessed without** having an **instance of a class**.
   - But if there are instances, a static variable of a class will be shared by all instances of that class; there is only one copy.
6. If we want all the instances of a class to **share data**, use static variables.
   - static variables stored values of a variables in a common memory location

Example 1: Static Variable

```
public class Kira {
   // Declare and initialize static variable
   static int kaunterKira = 0;

   public Kira() {
      kaunterKira += 1; // Modify the value in the constructor
   }

   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

      System.out.println (“Count of Kira is " + kaunterKira);
   }
}
```

Example 1: Static Variable

- In the preceding code, the `static kaunterKira` variable is **set to 0 when the `Kira` class is first loaded** by the JVM, **before any `Kira` instances are created**.
- Whenever a `Kira` instance is created, the `Kira` constructor runs and increments the static `kaunterKira` variable.
- When this code executes, **three `Kira` instances are created** in main(), and the result is:

```
Count of Kira is 3
```

Example 2: Using Instance Variable

```
public class Kira {
   int kaunterKira = 0;  // Declare and initialize instance variable

   public Kira() {
      kaunterKira += 1;  // Modify the value in the constructor
   }

   public static void main (String [] args) {
      new Kira();
      new Kira();
      new Kira();

      System.out.println ("Count of Kira is " + kaunterKira);
   }
}
```

Example 2: Using Instance Variable

- When this code executes, it should still create three **`Kira`** instances in main() , but the result is a **compilation error**
- Compilation completed with exit code: 1 error

```
Kira.java:14: error: non-static variable kaunterKira cannot be referenced from a static context
        System.out.println ("Kiraan Kira ialah " + kaunterKira);
                                                    ^
1 error
```

Static Modifier

- Things you **can** mark as static:
  - Methods
  - Variables
  - A class nested within another class, but not within a method
  - Initialization blocks

Static Modifier

- Things you **cannot** mark as static:
  - Constructors (makes no sense; a constructor is used only to create instances)
  - Classes (unless they are nested)
  - Interfaces
  - Method local inner classes (will cover in later chapter)
  - Inner class methods and instance variables
  - Local variables

Static Nested Class

- A static nested class is simply a class that's a static member of the enclosing class:

```
class BigOuter {
static class Nested { } }
```

- The static modifier in this case says that the nested class is **a static member of the outer class.**
  - That means it can be accessed, as with other static members, **without having an instance of the outer class.**

Using Static Nested Classes

- Use standard syntax to access a static nested class from its enclosing class.
- The syntax for instantiating a static nested class from a non-enclosing class is a little different from a normal inner class, and looks like this:

Using Static Nested Classes

STATIC VARIABLE & METHOD

- Use standard syntax to access a static nested class from its enclosing class.
- The syntax for instantiating a static nested class from a non-enclosing class is a little different from a normal inner class

```
class BigOuter {
    static class Nest {
      void go(){System.out.println("hi");}
      }
    }
class Broom {
    static class B2 {
        void goB2() {System.out.println("hi 2"); }
    }

    public static void main(String[] args) {
        BigOuter.Nest n = new BigOuter.Nest();
        n.go();
        B2 b2 = new B2(); // access the enclosed class
        b2.goB2();
    }
}
```

Static Variable

- Static variables store values for the variables in the common memory location.
- All object of the same class are affected if one object changes the value of a static variable.
- Public static variables are referred to using "**dot notation**" (if in other classes):

```
ClassName.staticVar
```

```
double area = Math.PI * r * r;
setBackground(Color.BLUE);
c.add(btn, BorderLayout.NORTH);
System.out.println(area);
```

Static `final`

- A static final can hold a constant shared by all objects of the class e.g.

```
public class RollingDie
{
    private static final double slowDown = 0.97;
    private static final double speedFactor = 0.04;
    ...
```

Reserved words: static final

Static Methods

- Static methods can access and manipulate a class's static fields.
- They cannot access non-static fields or call non-static methods of the class.
- They are called using "dot notation":

```
ClassName.staticMethod(...)
```

```
double x = Math.random();
double y = Math.sqrt (x);
System.exit();
```

Static Method: Example

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- The above program contains the **`main`** method and the **`max`** method (which is a static method) which demonstrates calling a method **`max`** to return the largest of the **`int`** values.
- In this example, the **`main`** method invokes `max(`**`i,j`**`)`, which is defined in the same class with the **`main`** method.

Static Method: Example (cont.)

pass the value of i

pass the value of j

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

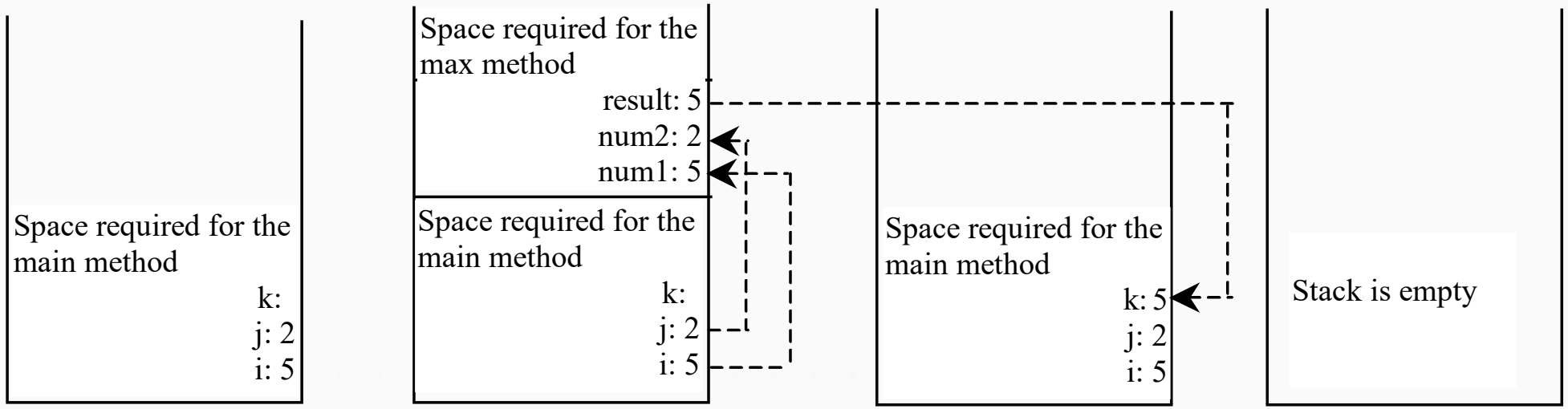
```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

- When the max method is invoked, variable **`i`**'s value 5 is passed to **`num1`**, and variable **`j`**'s value 2 is passed to **`num2`** in the **`max`** method.
- The flow of control transfers to the **`max`** method.
- The **`max`** method is executed. When the **`return`** statement in the **`max`** method is executed, the max method **`returns`** the control to its caller.

Call Stacks

- Each time **a method is invoked**, the system stores parameters and variables in an area of memory known as a **stack**, which stores elements in a last-in-first-out fashion.
- When a method **calls another method**, the caller's stack space is kept intact, and new space is created to handle the new method call.
- When **a method finishes its work** and **returns to its caller**, its associated space is released.

Call Stacks



The main method is invoked.

The max method is invoked.

The max method is finished and the return value is sent to k.

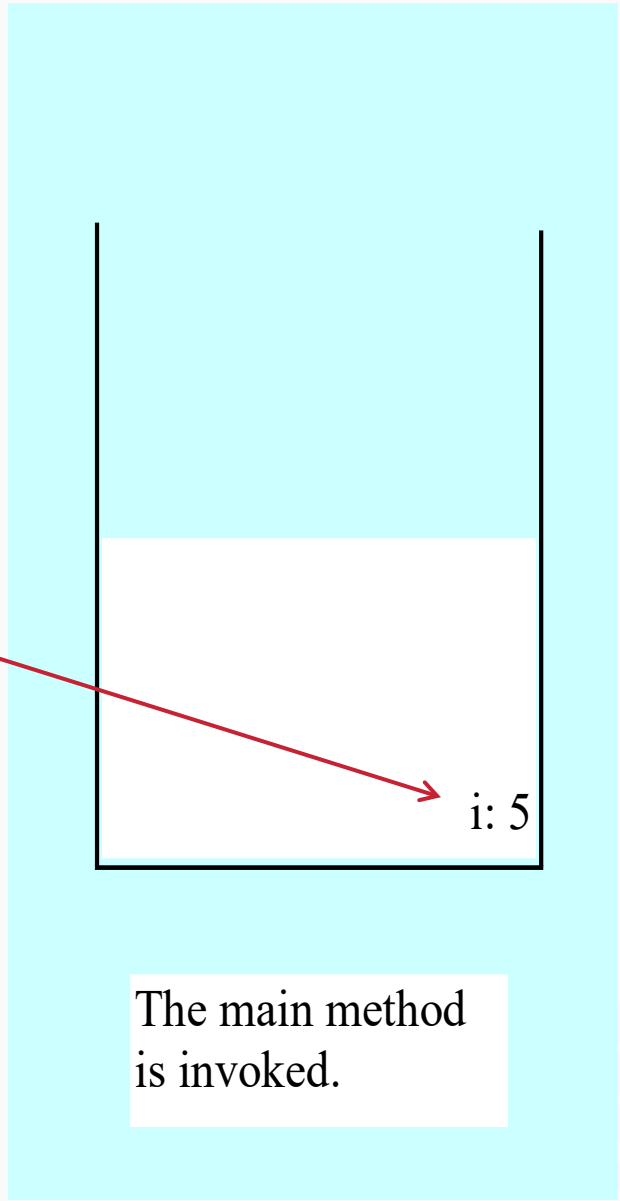
The main method is finished.

Call Stacks

i is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

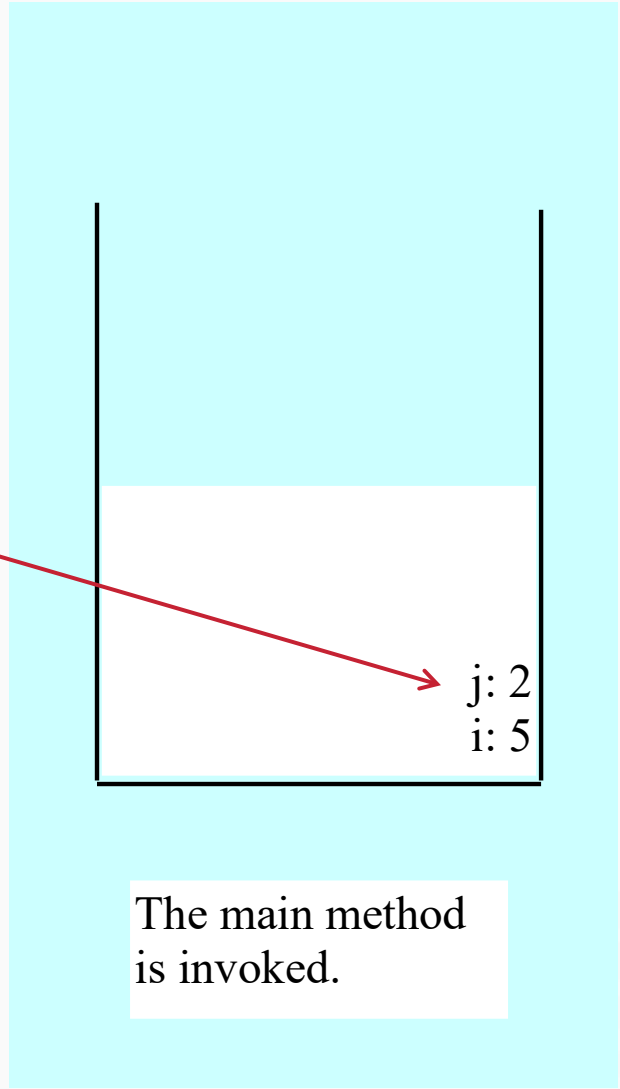


Call Stacks

j is declared and initialized

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

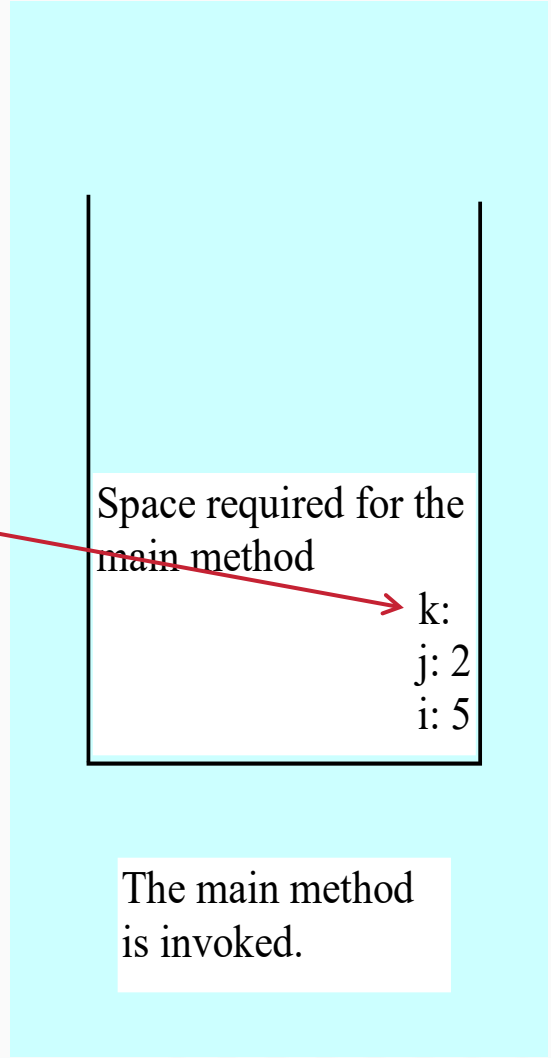


Call Stacks

declare k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

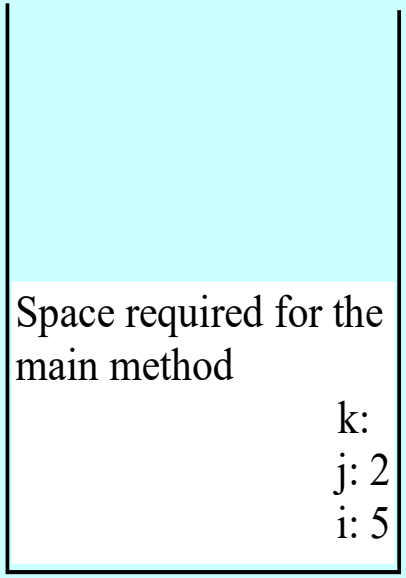


Call Stacks

Invoke max(i, j)

```
public static void main(Stri      rgs) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



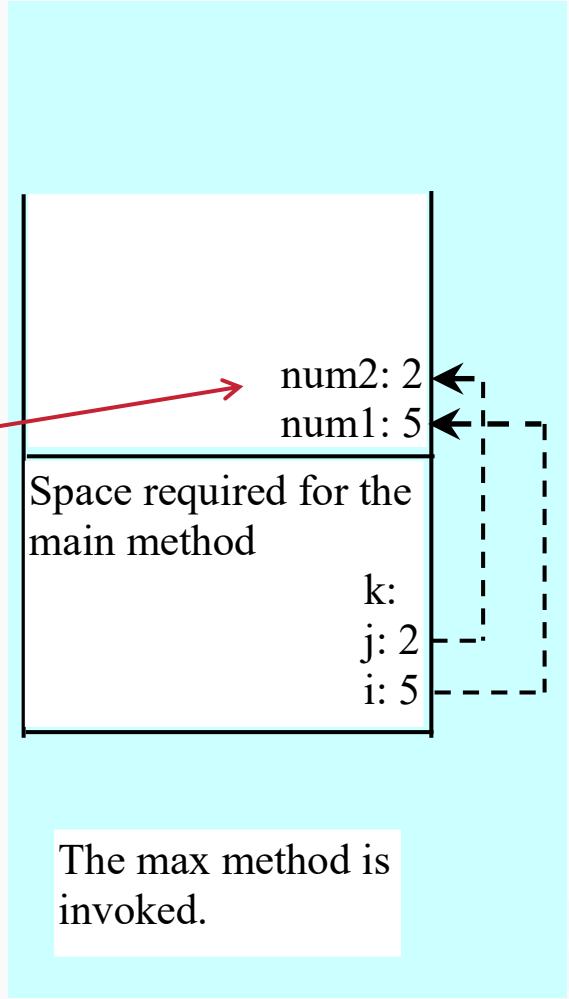
The main method is invoked.

Call Stacks

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

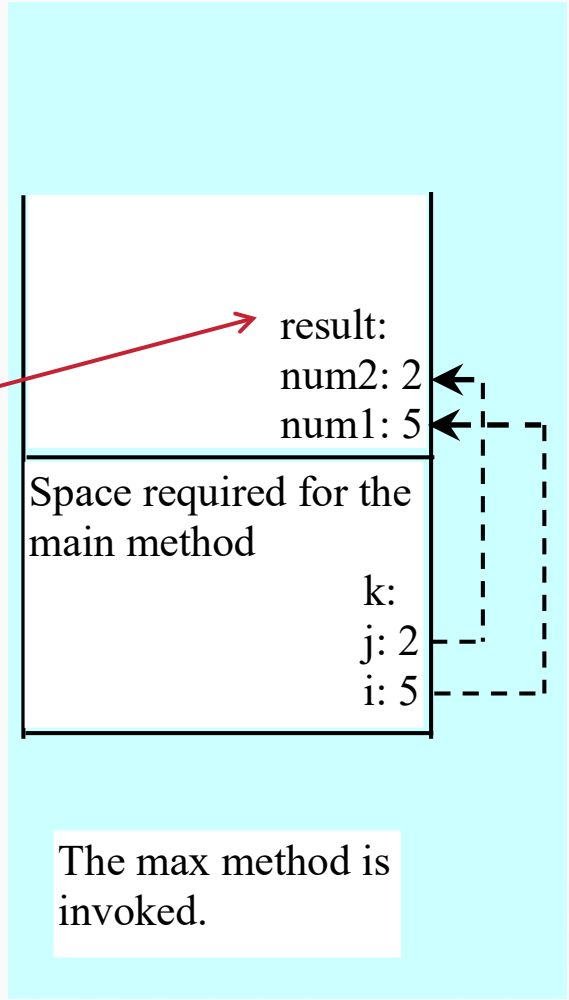


Call Stacks

pass the values of i and j to num1 and num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

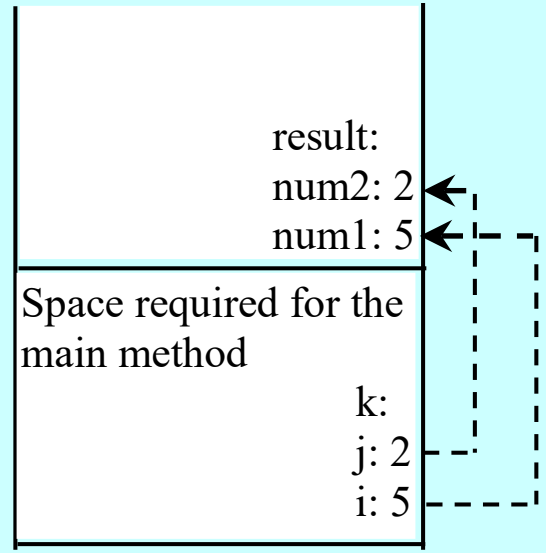


Call Stacks

(num1 > num2) is true

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



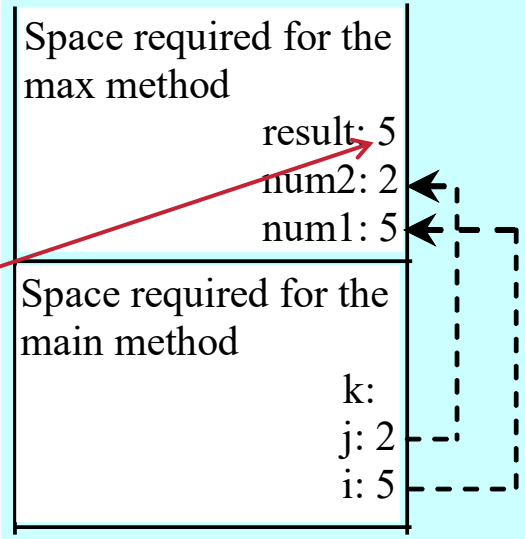
The max method is invoked.

Call Stacks

Assign num1 to result

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



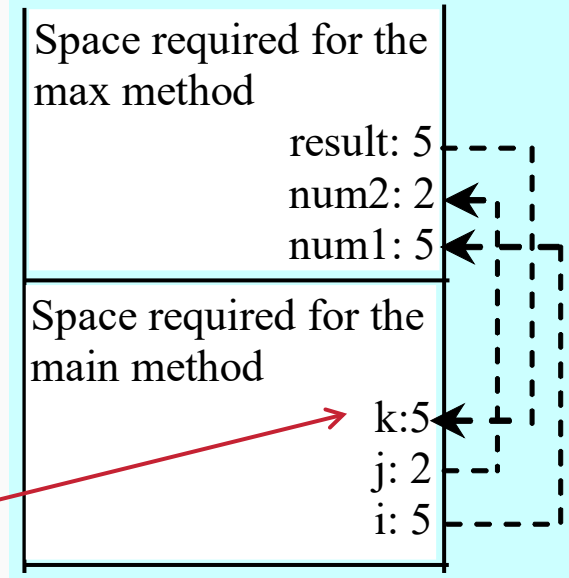
The max method is invoked.

Call Stacks

Return result and assign it to k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int nu    {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



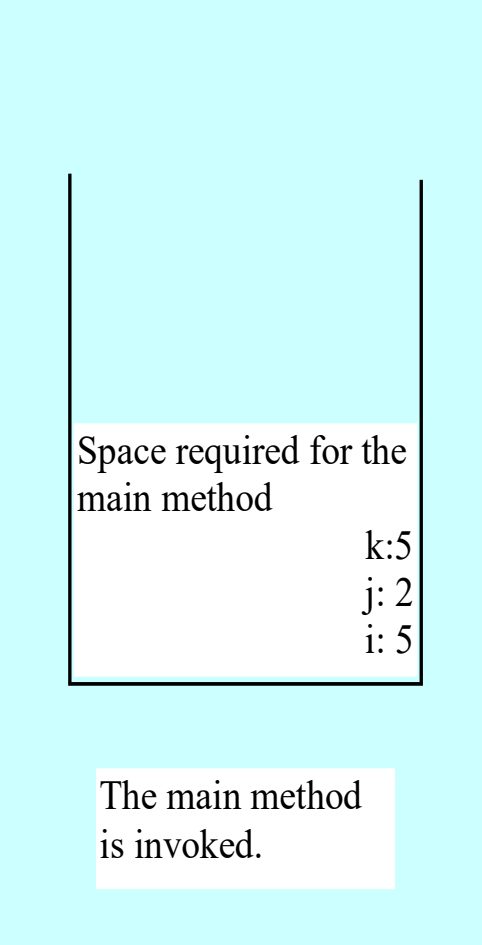
The max method is invoked.

Call Stacks

Return result and assign it to k

```
public static void main(String[] args)
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```



Static method cannot access an instance variable

- Accessing **`size`** (instance variable) from inside **`doMore`** static method

```
class Foo

int size = 42;
static void doMore( ){
  int x = size;
}
```

static method cannot access an instance (non-static) variable

Static method cannot access a non-static method

- Accessing **`go`** (non-static method) from inside **`doMore`** static method

```
class Bar

void go ( );
static void doMore( ){
    go( );
}
```

static method cannot access a non-static method

Static method can access a static method or variable

- Accessing **`woo`** (static method) and **`count`** (static variable) from inside **`doMore`** static method

```
class Baz

static int count;
static void woo( ){ }
static void doMore( ){
    woo( );
    int x = count;
}
```

static method *can* access a static method or variable

Scope of Local Variables

- A local variable: a variable **defined inside a method**.
- Scope: the part of the program where the variable can be referenced.
- The scope of a local variable **starts from its declaration** and **continues to the end of the block** that contains the variable.
- We can declare a local variable with the same name multiple times in different non-nesting blocks in a method, but we **cannot declare a local variable twice in nested blocks**.

Scope of Local Variables: Example “for” loop

- A variable declared in the initial action part of a **for loop header** has its scope in the **entire loop**.
- But a variable declared inside a **for loop body** has its scope limited in the loop body from its declaration and to the end of the block that contains the variable.

The scope of i

The scope of j

```
public static void method1() {
   .
   .
   for (int i = 1; i < 10; i++) {
      .
      .
      int j;
      .
      .
      .
   }
}
```

Scope of Local Variables (cont.)

It is fine to declare i in two non-nesting blocks

```
public static void method1() {
  int x = 1;
  int y = 1;

  for (int i = 1; i < 10; i++) {
    x += i;
  }

  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

It is wrong to declare i in two nesting blocks

```
public static void method2() {

  int i = 1;
  int sum = 0;

  for (int i = 1; i < 10; i++)
    sum += i;
  }

}
```

Scope of Local Variables (cont.)

```
// Fine with no errors
public static void correctMethod() {
  int x = 1;
  int y = 1;
  // i is declared
  for (int i = 1; i < 10; i++) {
    x += i;
  }
  // i is declared again
  for (int i = 1; i < 10; i++) {
    y += i;
  }
}
```

```
// With errors
public static void incorrectMethod() {
  int x = 1;
  int y = 1;
  for (int i = 1; i < 10; i++) {
    int x = 0;
    x += i;
  }
}
```

Method Overloading

- Method overloading is **two or more methods have the same name** but different parameter lists within one class.
- The Java compiler determines which method is used based on the **method signature**.
- Overloaded methods **must have different parameter lists**.
- Cannot overload methods based on different modifiers or return type.

Overloading Methods: Example

```
public static int max(int num1, int num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}
public static double max(double num1, double num2) {
  if (num1 > num2)
    return num1;
  else
    return num2;
}
public static double max(double num1, double num2, double num3) {
  return max(max(num1, num2), num3);
}
```